OTA 升级场景 Watchdog 超时复现测试报告

- 日期：2026-03-25
- 设备：Qualcomm SA8395P (Yocto + Linux PREEMPT_RT)
- 连接方式：adb shell
- 关联分析文档： `content/others/zstdext-ota-write-optimization.md`

一、测试背景

OTA 升级期间， `zstdext` 工具以 buffered I/O 方式向块设备写入大量数据后调用 `fsync()` ，导致脏页集中回写到 UFS 存储，引发 UFS IRQ handler 长时间持有 BH 锁，进而造成 `pmonitor` 检测到多个关键进程心跳超时（watchdog timeout）。

本测试目的：**在不依赖真实 OTA 包的情况下，模拟 zstdext 的 I/O 模式，复现 watchdog 超时现象**。

二、测试环境

项目	值
SoC	Qualcomm SA8395P
内核	Linux PREEMPT_RT
存储	UFS (MCQ 模式，5 个队列 IRQ 297-301)
当前启动槽位	`_a` （写入目标为 `_b` 分区，安全）
写入目标分区	`/dev/disk/by-partlabel/la_super_b`
dirty_ratio	20% (~2.6GB)
dirty_background_ratio	10%
watchdog 超时阈值	pmonitor 3000ms / updatemgr 内部 1000ms

关键进程状态（测试前确认均在运行）：

进程	PID
pmonitor	5572
ivcd	2084
mcd	2093
audiomgr	2078
updatemgr	5561

三、测试方法

测试脚本

见 `wd_reproduce_test.sh` 。

核心原理

模拟 `zstdext --seg-size=256` 的 I/O 模式：向 `_b` 备用分区进行大块 buffered write，然后调用 `sync` 触发脏页集中回写到 UFS，制造与真实 OTA 相同的 I/O 风暴。

测试步骤

1. **安全检查**：确认当前启动槽位为 `_a` ，目标 `_b` 分区未挂载
2. **采集基线**：记录 10 秒正常状态下的 DLT 日志，统计心跳和超时消息数量
3. **配置 ftrace**：启用 `sched_switch` 事件，buffer 256MB，用于事后分析调度延迟
4. **启动并行监控**：
   - DLT 实时采集（ `dlt-receive -a localhost` ）
   - 系统指标每 500ms 采样（上下文切换数、UFS IRQ 计数）
5. **执行 I/O 风暴**（4 个场景）：
   - **场景 A**：单次 256MB buffered write → sync
   - **场景 B**：预填零 256MB + 覆写 256MB → sync（模拟 zstdext 预填零 + 解压写入）
   - **场景 C**：1GB 连续 buffered write → sync
   - **场景 D**：连续 4 轮 256MB write + sync（模拟完整 OTA 多 segment）
6. **收集结果**：保存 ftrace、DLT 日志、dmesg、系统指标

执行命令

```
# 推送脚本到设备
adb push wd_reproduce_test.sh /tmp/
adb shell "chmod +x /tmp/wd_reproduce_test.sh"

# 执行测试
adb shell "/tmp/wd_reproduce_test.sh"
```

四、测试结果

4.1 Watchdog 超时复现成功

DLT 日志中捕获到以下 watchdog 超时告警：

```
2026/03/25 05:58:58.233601 ECU1 updm [UPDATE_MAIN]updatemgr watchdog timeout, will exit
2026/03/25 05:58:58.233745 ECU1 updm [UPDATE_MAIN_MATRIX_02]updatemgr watchdog timeout,
2026/03/25 05:58:57.789140 ECU1 rpcd MCU message queue timeout
```

`updatemgr` 的 `UPDATE_MAIN` 和 `UPDATE_MAIN_MATRIX_02` 两个线程触发了 watchdog 超时。

4.2 关键指标

指标	值	说明
I/O 总耗时	22 秒	4 个场景总计
UFS IRQ 增量	6524	测试期间 UFS 中断总增量
DLT timeout 告警	4 条	基线 0 条
UFS IRQ 线程最长单次运行	8 ms	Path 1 (BH 锁) 未直接触发
updatemgr 最大调度间隔	**1802.7 ms**	正常值 500ms，超标 3.6 倍
mcd 最大调度间隔	623 ms	
audiomgr 最大调度间隔	601 ms	
pmonitor 最大调度间隔	430 ms	
ktimers/5 D 状态总时长	**~1431 ms**	420ms + 1011ms 两段

4.3 ftrace 时间线分析

以下时间线从 ftrace 的 `sched_switch` 事件中提取，精确还原了超时发生的过程：

```
时间 (秒)          事件
──────────────────────────────────────────────────────────
3984.442991        updatemgr(5561) 正常唤醒，设置 500ms soft hrtimer 后 sleep
                   → 预期下次唤醒：~3984.943

3984.806268        ktimers/5 仍在正常处理定时器
                   CPU5 上 dd 写入活跃，UFS IRQ 频繁触发

3984.815890        ★ ktimers/5 进入 D 状态（不可中断睡眠）
                   被内核 writeback 路径的锁阻塞
                   |
                   | 420 ms 阻塞
                   ↓
3985.235613        ktimers/5 短暂恢复，但再次进入 D 状态
                   |
                   | 1011 ms 阻塞
                   ↓
3986.245715        ktimers/5 恢复正常，处理积压的 soft hrtimer
3986.245721        updatemgr(5561) 终于被唤醒
                   → 实际间隔：1802.7 ms（预期 500ms）
                   → 心跳间隔远超 1000ms 阈值
                   → pmonitor 判定 watchdog timeout
```

updatemgr 正常调度模式（500ms 周期，精度 < 0.1ms）：

```
切入: 3982.442715 (间隔 500.1 ms)
切入: 3982.942782 (间隔 500.1 ms)
切入: 3983.442851 (间隔 500.1 ms)
切入: 3983.942918 (间隔 500.1 ms)
切入: 3984.442991 (间隔 500.1 ms)
*** 异常间隔: 1802.7 ms ***
切入: 3986.245721 (间隔 1802.7 ms)   ← 触发 watchdog timeout
切入: 3986.745791 (间隔 500.1 ms)    ← 恢复正常
切入: 3987.245877 (间隔 500.1 ms)
```

4.4 失效路径确认

本次测试复现的是文档中描述的 **路径二：Soft hrtimer 延迟**。

```
dd buffered write + sync → UFS I/O 密集
  → writeback 路径持锁 → ktimers/5 进入 D 状态（被锁阻塞 1431ms）
  → updatemgr 的 soft hrtimer 无法被及时处理
  → 500ms 的 msgque_recv_wait 实际阻塞 1803ms
  → 心跳间隔远超 1000ms 阈值
  → pmonitor 判定 watchdog timeout
```

关键证据：

- `ktimers/5` （RT 优先级 98）进入 **D 状态**，说明它被内核锁阻塞而非被调度抢占
- updatemgr 是 `SCHED_NORMAL` （prio=120），其定时器走 soft hrtimer 路径，依赖 `ktimers/N` 线程处理
- 在 `ktimers/5` 阻塞期间，CPU5 上 UFS IRQ 线程（ `irq/302-qcom-mc` ）和 `dd` 进程交替运行

五、根因分析：内核/驱动锁设计缺陷，非 I/O 负载问题

5.1 不是写入太多太快

仅用 `dd if=/dev/zero` 写入不到 1GB 数据即触发了 watchdog 超时。这是一个极其普通的 I/O 操作，任何 Linux 系统都应无障碍处理。真实 OTA 场景（zstdext 写入几 GB + 预填零双倍脏页 + 解压 CPU 开销）只是让问题更容易触发，但不是根因。

5.2 `ktimers/5` （RT prio=98）被内核锁阻塞 1.4 秒

`ktimers/5` 是 RT 优先级 98 的内核定时器基础设施线程，负责处理该 CPU 上所有 `SCHED_NORMAL` 进程的 soft hrtimer。它进入 D 状态（不可中断睡眠）意味着被一把内核锁卡住——这不是调度优先级问题，是 **锁设计缺陷**。

```
3984.815890  ktimers/5  prev_state=D  → 进入不可中断睡眠（被锁阻塞）
             ... 420ms 完全阻塞 ...
3985.235613  ktimers/5  prev_state=D  → 短暂醒来但再次被锁阻塞
             ... 1011ms 完全阻塞 ...
3986.246657  ktimers/5  prev_state=S  → 终于恢复正常
```

5.3 UFS IRQ handler 自身也在 D 状态中反复挣扎

trace 显示 3985.235 期间 CPU5 上 `irq/302-qcom-mc` （UFS IRQ 线程，prio=49）反复进入 D 状态：

```
irq/302-qcom-mc [005] D..23  3985.235005: prev_state=D ==> swapper
          idle  [005]        3985.235023:              ==> irq/302-qcom-mc
irq/302-qcom-mc [005] D..23  3985.235031: prev_state=D ==> kworker
                             ... 每 ~45μs 重复一次 ...
irq/302-qcom-mc [005] D..23  3985.235402: prev_state=D ==> swapper
                             ... 持续至 ...
irq/302-qcom-mc [005] D..23  3985.235611:              ==> ktimers/5 (终于唤醒 ktimers)
       ktimers/5 [005] d..21  3985.235613: prev_state=D ==> kworker  (但 ktimers 立刻又被
```

UFS IRQ handler 反复：醒来 → 尝试获取锁 → 拿不到 → D 状态 → IPI 唤醒 → 再试... 每次只运行 ~10-15μs。**它和 `ktimers/5` 被同一把锁阻塞**。

5.4 根因：PREEMPT_RT 锁转换 × UFS 驱动 IRQ 注册方式冲突

```
PREEMPT_RT 内核特性:
├── spinlock → rt_mutex（可睡眠）
├── local_bh_disable() → 获取 per-CPU BH 锁
└── request_irq() 的 handler → irq_forced_thread_fn() 包裹
    └── 在 local_bh_disable/enable 之间执行（持有 BH 锁）

UFS MCQ 驱动 (ufs-qcom.c):
├── 使用 request_irq() 注册 qcom-mcq-esi handler
├── PREEMPT_RT 下被强制线程化 → irq_forced_thread_fn()
├── handler 内部处理 I/O 完成 → 涉及 folio writeback 锁
└── folio writeback 锁在 PREEMPT_RT 下也是 rt_mutex（可睡眠）

冲突链:
├── UFS IRQ handler 持有 BH 锁 → 进入 writeback 路径 → 等待 folio 锁 → D 状态
├── ktimers/N 需要处理 soft hrtimer → 需要进入 softirq 上下文 → 等 BH 锁 → D 状态
└── 两者互相等锁 → priority inversion → RT 定时器线程被阻塞 1.4 秒
```

5.5 对比：正常 Linux 内核 vs PREEMPT_RT 内核

	标准 Linux (非 RT)	此系统 (PREEMPT_RT)
spinlock	自旋，不睡眠	转为 rt_mutex，可睡眠

	标准 Linux (非 RT)	此系统 (PREEMPT_RT)
local_bh_disable	关闭软中断，原子操作	获取 per-CPU BH 锁（可阻塞）
UFS IRQ handler	硬中断/softirq 上下文，快速完成	被 `irq_forced_thread_fn` 线程化，持有 BH 锁
hrtimer 处理	硬中断上下文，微秒级	soft hrtimer 走 `ktimers/N` ，可被锁阻塞
1GB dd 写入	**正常，无延迟**	**ktimers 被锁阻塞 1.4s → watchdog 超时**

5.6 总结

问题性质：内核/驱动层面的锁设计缺陷，不是应用层 I/O 负载问题。

UFS MCQ 驱动使用 `request_irq()` 而非 `request_threaded_irq()` 注册中断，导致在 PREEMPT_RT 下被 `irq_forced_thread_fn` 包裹，handler 全程持有 BH 锁。当 handler 内部触碰 writeback 路径的可睡眠锁时，BH 锁长时间不释放， `ktimers/N` RT 线程被 BH 锁阻塞，无法处理 soft hrtimer，所有依赖 soft hrtimer 的 `SCHED_NORMAL` 线程定时器失效，最终导致 watchdog 超时。

文档 4.7 节提出的修复方案（改用 `devm_request_threaded_irq` + `IRQF_ONESHOT` 消除 BH 锁包裹）是从根本上解决此问题的正确方向。

六、结论

1. **复现成功**：仅用 `dd if=/dev/zero` + `sync` 即可触发 `updatemgr` watchdog 超时，无需真实 OTA 包
2. **触发阈值低**：dd 写入 /dev/zero 的 I/O 压力远低于真实 zstdext OTA 场景（无解压 CPU 开销、无预填零双倍脏页），但已足以触发超时，说明问题不在 I/O 负载量
3. **根因确认**：UFS MCQ 驱动的 IRQ 注册方式与 PREEMPT_RT 内核的锁转换机制冲突，导致 BH 锁 → folio writeback 锁 → priority inversion， `ktimers/N` RT 线程被阻塞长达 1.4 秒
4. **与文档分析一致**：复现路径与 `zstdext-ota-write-optimization.md` 中描述的 Path 2 (Soft hrtimer 延迟) 完全吻合
5. **优化方向**：
   - **根治**（内核层）：修改 UFS 驱动使用 `devm_request_threaded_irq` + `IRQF_ONESHOT` ，消除 `irq_forced_thread_fn` 的 BH 锁包裹
   - **缓解**（应用层）：将关键线程提升为 `SCHED_FIFO` （使用 hard hrtimer 替代 soft hrtimer，绕开 `ktimers/N` 被阻塞的问题）

六、文件清单

```
ota-watchdog-test/
├── ota-watchdog-reproduce-report.md   # 本报告
├── wd_reproduce_test.sh               # 测试脚本
└── data/
    ├── dlt_baseline.txt               # 10 秒基线 DLT 日志
    ├── dlt_during_io.txt              # I/O 期间 DLT 日志（含 watchdog timeout 告警）
    ├── dmesg_tail.txt                 # 内核日志尾部
    ├── dd_output.txt                  # dd 执行输出
    ├── pids.txt                       # 关键进程 PID 记录
    └── sys_metrics.csv                # 系统指标时间序列（上下文切换、UFS IRQ 计数）
    └── trace.txt                      # ftrace文件
```